Escola Secundária Miguel Torga

Curso Técnico de Gestão de Equipamentos Informáticos

Projeto Stable Pack API

Estudante: Gabriel Alexandre Rocha

Data:1 Março de 2026

Ano letivo: 2025/2026

Agradecimentos

Agradeço a todos os professores que me acompanharam ao longo destes anos, que auxiliaram e contribuíram para a minha evolução durante o curso, em particular ao Diretor do Curso Profissional Curso Técnico de Gestão de Equipamentos Informáticos e orientador deste projeto, professor Ricardo Pires e ao professor João Monteiro, que acompanhou o desenvolvimento desta Prova de Aptidão Profissional.

Resumo

Este projeto foi desenvolvido no âmbito da Prova de Aptidão Profissional do Curso Profissional de Técnico de Gestão de Equipamentos Informáticos e consiste na implementação de uma API destinada a suportar uma aplicação móvel inspirada em plataformas de entregas.O sistema permite a gestão integrada de contas de clientes, disponibilizando funcionalidades tais como registo, autenticação, recuperação de password de forma totalmente segura. Para garantir uma maior segurança e fiabilidade dos utilizadores deste sistema foram utilizadas tecnologias como FastAPI para construção da API, bcrypt para encriptação de passwords e JWT para gestão de sessões e autenticação. O relatório detalha o processo de desenvolvimento, a arquitetura da API, os modelos de dados utilizados e os principais mecanismos implementados para assegurar a integridade e correto funcionamento da aplicação.

Índice

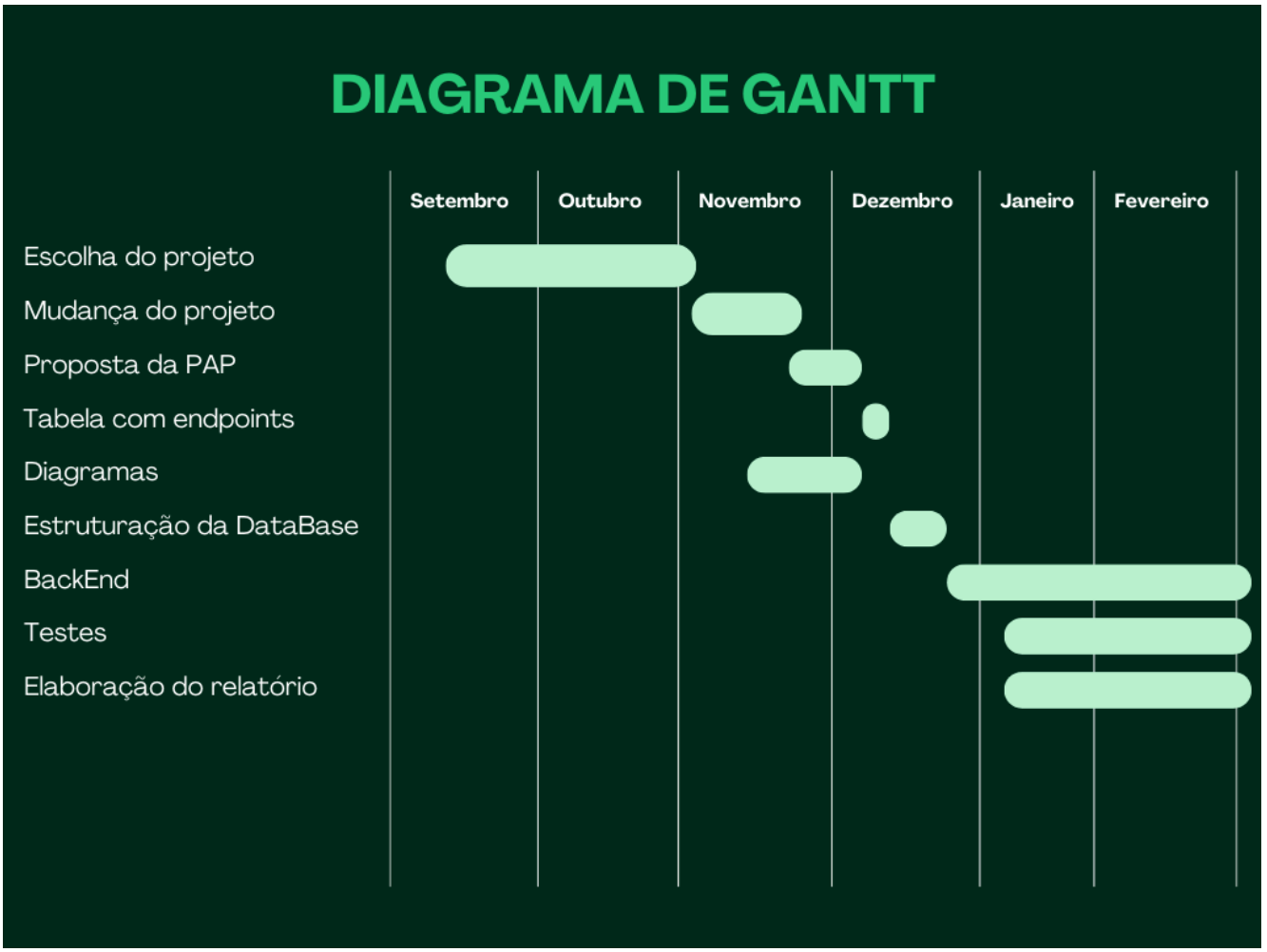
Agradecimentos **2**
Resumo **3**
Índice **4**
Introdução **6**
Cronograma **7**
Enquadramento Teórico **7**
1. Conceito de API 7
2. Arquitetura REST 8
3. FastAPI 8
4. Base de Dados (MySQL) 8
5. Encriptação de Passwords com bcrypt 8
6. Autenticação com JWT 9
7. Envio de Emails com SMTP 9
8. Cloudflare Tunnel e CGNAT 9
Desenvolvimento do projeto **9**
1. Início do projeto 9
2. Planeamento e Organização 10
3. Escolha das tecnologias 10
4. Desenvolvimento da Estrutura da API 11
5. Desenvolvimento dos ficheiros 12
6. Estruturação da Base de Dados 12
7. Acesso Externo com Cloudflare Tunnel 13
8. Testes e Validação 13
Implementação Técnica da API **13**
1. Importação das Bibliotecas 13
2. Criação do Router 14
3. Modelos de Dados 14
4. Endpoint de Registo 14
5. Endpoint de Recuperação de Password 15
6. Endpoint de Redefinição de Password 15
7. Endpoint de Login 16
Conclusão **16**
Bibliografia **17**
Anexos **17**

Introdução

O presente projeto foi desenvolvido no âmbito da prova de aptidão profissional do Curso Profissional Técnico de Gestão de Equipamentos Informáticos e integra o trabalho realizado para a criação de uma startup em desenvolvimento, cujo objetivo é criar uma solução tecnológica para o setor de entregas de alimentos combinando com uma aplicação móvel.

A componente desenvolvida neste projeto corresponde à API principal da plataforma, responsável pela autenticação, segurança e comunicação. A API foi construída com recurso ao FastAPI, utilizando MySQL como base de dados, bcrypt para encriptação de passwords e JWT para gestão de sessões e autenticação seguras assim como smtplib para envio dos emails de recuperação. Para permitir a utilização de usuários fora da rede local, foi utilizado também o sistema de túneis da cloudflare para permitir que dispositivos fora da rede local acedam à API. Entre as funcionalidades desenvolvidas destacam-se o registo de utilizadores, login, recuperação de password através de *tokens* temporários, redefinição de credenciais e emissão de *tokens* de acesso.
Esta API constitui o núcleo lógico da aplicação, servindo de ponte entre o *frontend*, Base de dados,desenvolvido pela Prova de Aptidão Profissional de um elemento pertencente à startup.

Cronograma



Anexo 1 - Diagrama de Gantt

Enquadramento Teórico

1. Conceito de API

A sigla API refere-se a “Application Programming Interface” e representa um conjunto de diretrizes e pontos de extremidade que possibilitam a ligação entre diferentes componentes de um sistema, como o *frontend* e o *backend*. Neste cenário, a API atua como um intermediário entre a aplicação móvel e o banco de dados, recebendo solicitações, manipulando-as e retornando respostas. Geralmente, as APIs contemporâneas adotam o formato REST, que emprega métodos como: HTTP (GET, POST, PUT, DELETE) para efetuar as operações.

2. Arquitetura RESTful

RESTful é uma abordagem de arquitetura que se refere a “Transferência de Estado Representacional” e apresenta:

Stateless - Cada requisição é autônoma e inclui informações

Utilização de Métodos HTTP - Get para obter informações, POST para fazer criações, PUT para modificações, DELETE para exclusões

Respostas em JSON - Formato utilizado nas respostas de requisições

Divisão entre cliente e servidor - o *frontend* e o *backend* operam de maneira separada, não dependendo um do outro para funcionar.

3. FastAPI

O FastAPI representa um novo e atual framework em Python voltado para a criação de APIs seguras e assíncronas de maneira eficiente, simplificando essa tarefa. Devido a esses fatores, um grande número de usuários está a utilizar o FastAPI nos dias de hoje, sendo algumas das suas principais características:

- Elevada performance
- Validação de dados com suporte nativo por intermédio do Pydantic
- Criação automática do endpoint /docs utilizando redoc e swagger
- Fácil integração com JWT, bases de dados e muito mais.

4. Base de Dados (MySQL)

O MySQL é um sistema conhecido por gerenciar bancos de dados relacionais e é muito popular por sua confiabilidade, eficiência e fácil integração. Neste projeto, o MySQL desempenha uma função crucial ao guardar informações sobre usuários, senhas criptografadas, *tokens* temporários e perfis, entre outras informações.

5. Encriptação de Passwords com bcrypt

A proteção das senhas constitui um dos aspetos mais essenciais de qualquer sistema. Por essa razão, este projeto utiliza o bcrypt, que é um método de hashing que criptografa as senhas de todos os utilizadores de maneira que

não pode ser revertida, assim sendo, mesmo se ocorrer uma intrusão na base de dados, as senhas permanecerão em criptografadas.

6. Autenticação com JWT

O JWT representa uma abordagem segura e contemporânea para a autenticação de sessões, amplamente adotada por quase todas as organizações. Um Token JWT inclui a identificação do usuário, endereço de e-mail, função e data de expiração. O JWT não requer a manutenção de sessões no servidor da API, é simples de validar e proporciona segurança quando é assinado com uma chave secreta.

7. Envio de Emails com SMTP

SMTP é uma biblioteca padrão do python que possibilita o envio de mensagens eletrônicas por meio de servidores SMTP, neste projeto é utilizada para enviar emails aos usuários, permitindo que eles recebam o código e possam modificar a senha caso a esqueçam.

8. Cloudflare Tunnel e CGNAT

Na fase de testes, enfrentei um desafio: a API operava somente em modo local, porém eu necessitava de transformar essa ligação numa ligação externa permanente para ligar ao *backend*. Depois de investigar formas de obter um IP fixo sem altos custos, encontrei o Cloudflare Tunnel como solução para esse problema.

Cloudflare Tunnel cria uma ligação protegida entre o servidor local e o servidor da Cloudflare, permitindo o acesso externo sem precisar abrir portas do roteador. A proteção do servidor da API é reforçada, uma vez que ele não se conecta diretamente ao computador, mas sim ao servidor da Cloudflare, que em seguida se comunica com a máquina, evitando assim potenciais ataques.

Desenvolvimento do projeto

1. Início do projeto

A concepção do desenvolvimento deste projeto teve início com o surgimento da startup solvex, que se dedica à elaboração de uma API para um aplicativo de entrega de comidas. O foco primordial é implantar uma API que seja segura, expansível e eficaz, permitindo autenticação de maneira protetiva, ligação com o *frontend* e recuperação de password

No começo, havia uma proposta distinta para a PAP, mas após uma avaliação, foi decidido que era mais coerente integrar o projeto com as atividades realizadas na startup dentro da PAP, considerando que iríamos desenvolver nossa startup nesse período.

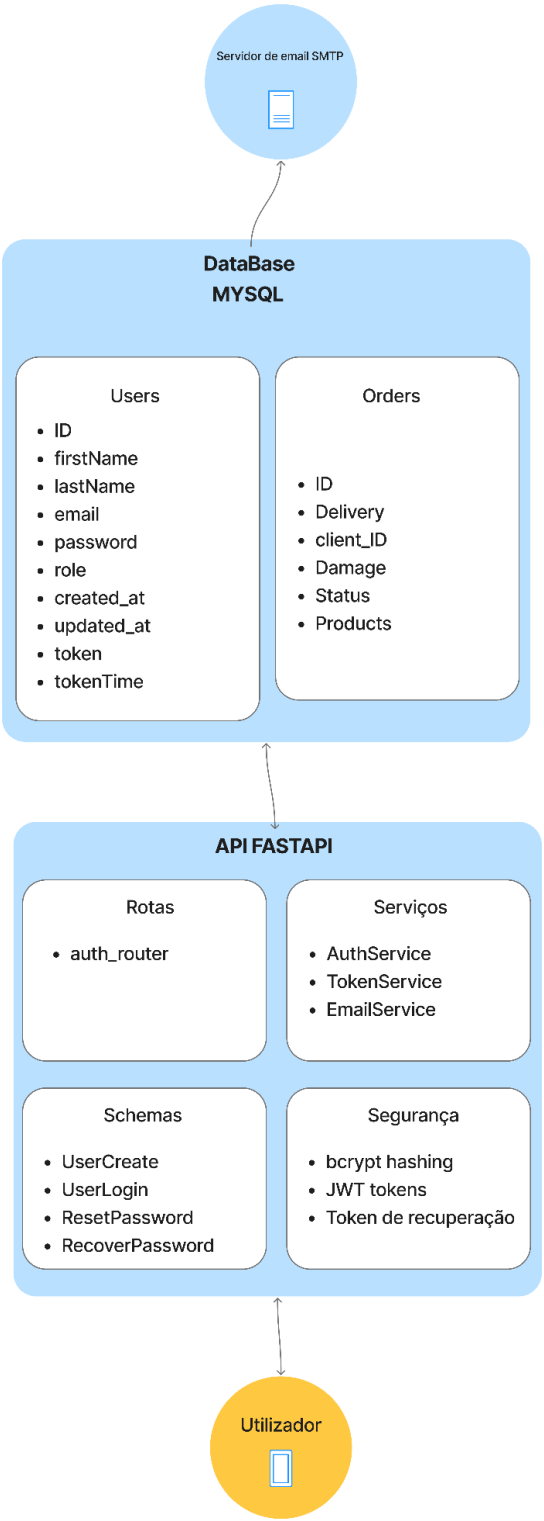
2. Planeamento e Organização

Para garantir a realização deste projeto as partes do desenvolvimento foram distribuídas em um diagrama de Gantt estas partes como:

- Escolha e reformulação do projeto
- Elaboração da proposta da PAP
- Criação da tabela de endpoints
- Desenvolvimento dos diagramas
- Estruturação da base de dados
- Desenvolvimento do backend
- Testes e validação
- Elaboração do relatório

3. Escolha das tecnologias

escolha do FastAPI foi baseada na sua agilidade, facilidade de uso e documentação gerada automaticamente. O MySQL foi selecionado pela sua confiabilidade, facilidade de uso e familiaridade. O bcrypt foi adotado para assegurar o hash seguro das senhas. O JWT foi escolhido para uma autenticação moderna e segura. O SMTP é utilizado para o envio de e-mails. O Cloudflare Tunnel foi implementado para possibilitar o acesso externo sem a necessidade de desbloquear portas no router.



Anexo 2 - Diagrama de Arquitetura do Sistema

4. Desenvolvimento da Estrutura da API

A API foi organizada em ficheiros independentes, facilitando a leitura, manutenção e escalabilidade do código.

Nome	Estado	Data de modificação	Tipo	Tamanho
.venv		06/02/2026 17:22	Pasta de ficheiros	
__pycache__		27/02/2026 18:01	Pasta de ficheiros	
Documentos		01/03/2026 14:42	Pasta de ficheiros	
Relatório		06/02/2026 17:21	Pasta de ficheiros	
.env		26/02/2026 23:01	Ficheiro ENV	1 KB
.gitignore		09/02/2026 08:34	Arquivo Fonte Git ...	1 KB
auth_routes.py		26/02/2026 22:35	Arquivo Fonte Pyt...	5 KB
database.py		22/02/2026 15:32	Arquivo Fonte Pyt...	1 KB
email_recover.py		22/02/2026 15:58	Arquivo Fonte Pyt...	2 KB
jwt_handler.py		26/02/2026 22:06	Arquivo Fonte Pyt...	1 KB
main.py		26/02/2026 10:04	Arquivo Fonte Pyt...	1 KB
requirements.txt		01/03/2026 20:11	Arquivo Fonte Text	3 KB

Anexo 3 - Estrutura dos ficheiros

5. Desenvolvimento dos ficheiros

.venv - É um Diretório que contém um ambiente virtual do python, incluindo todas as bibliotecas utilizadas neste projeto.

pycache - Pasta criada automaticamente pelo Python para armazenar ficheiros compilados permitindo uma execução mais rápida do código.

.env - Ficheiro onde são guardadas variáveis sensíveis, como palavras-passe, credenciais e chaves secretas.

requirements.txt - Lista de todas as bibliotecas necessárias para executar a API, permite instalar rapidamente todas as dependências.

main.py - É responsável pela inicialização da API, nele são configuradas rotas.

database.py - Este ficheiro contém a função get_database( ),responsável por estabelecer a ligação com a base de dados.

jwt_handler.py - Implementa a criação de *tokens* JWT utilizados no processo de autenticação, gera *tokens* assinados com chave secreta e tempo de expiração.

email_recover.py - Responsável por enviar os emails de recuperação e utiliza o protocolo SMTP. É executado em segundo plano para não atrasar

auth_routes.py - Ficheiro principal da API no que diz respeito à autenticação.

6. Estruturação da Base de Dados

A base de dados foi desenvolvida no MySQL, incluindo a base pap juntamente com as tabelas users e orders. A tabela users armazena todas as informações essenciais para a autenticação e administração de usuários. O design foi concebido com a intenção de facilitar a expansão futura, de modo a permitir a integração de restaurantes, entregadores e pedidos.

7. Acesso Externo com Cloudflare Tunnel

Durante a elaboração deste projeto, enfrentou-se a dificuldade de empregar a minha API em um aparelho que estava fora da minha rede local. Para resolver esse impasse, foi introduzido o Cloudflare Tunnel, que estabelece uma conexão segura entre o servidor local e a Cloudflare, facilitando o acesso externo sem revelar qualquer dado do servidor da API.

8. Testes e Validação

Os testes foram executados utilizando o aplicativo Postman e a ferramenta do FastAPI /docs, a qual apresenta um método de teste que nos permite experimentar. Testes também foram conduzidos com a interface *frontend*.

Implementação Técnica da API

1. Importação das Bibliotecas

Estas foram as bibliotecas utilizadas em auth_routes.py para garantir a autenticação

fastapi - É a biblioteca da nossa framework, e dentro dela existem as bibliotecas APIRouter que permite organizar os nossos endpoints, HTTPException para criar exceções, status contêm códigos de HTTP, e BackgroudTasks que permite executar tarefas em se gundo plano neste caso está a ser utilizado para enviar emails
pydantic - usado para validar dados
database - Vai buscar a conexão da função get_database em database.py e retorna a conexão da base de dados
send_recovery_email - Função que permite enviar os emails de recuperação
create_access_token - Gera os TokensJWT para permitir que os utilizadores iniciem sessão.
bcrypt - Encriptação de passwords.
secrets - Geração de *tokens* seguros-
datetime - Manipulação de datas
pymysql - Comunicação com MySQL

```
from fastapi import APIRouter, HTTPException, status, BackgroundTasks
from pydantic import BaseModel
from database import get_database
from email_recover import send_recovery_email
from jwt_handler import create_access_token
import bcrypt
import secrets
import datetime
import pymysql
```

Anexo 4 - Bibliotecas Utilizadas

2. Criação do Router

auth_route define o prefixo e organiza os endpoints relacionados com autenticação contidos no ficheiro. Exemplo: /auth/…

```
auth_route = APIRouter(prefix="/auth", tags=["Auth"])
```

Anexo 5 - auth_route

3. Modelos de Dados

Este modelo de dados define o formato obrigatório dos pedidos, por exemplo para o endpoint /auth/Register vai precisar obrigatoriamente de firstName, lastName, email, password, role.

```
class RegisterRequest(BaseModel):
    firstName: str
    lastName: str
    email: str
    password: str
    role: str

class LoginRequest(BaseModel):
    email: str
    password: str

class RecoverPassword(BaseModel):
    email: str
class ResetPassword(BaseModel):
    email: str
    password: str
    token: str
```

Anexo 6 - Pydantic

4. Endpoint de Registo

Este Endpoint vai receber os dados obrigatórios do pedido e vai verificar se o email já existe na base de dados caso exista dá erro caso não exista irá encriptar a password com bcrypt e insere um novo utilizador na base de dados e devolve uma mensagem de sucesso.

```python
@auth_route.post("/register")
async def register(request: RegisterRequest):
    db = get_database()
    cursor = db.cursor()

    cursor.execute("SELECT * FROM users WHERE email = %s", (request.email,))
    user = cursor.fetchone()

    if user:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Email já foi registado")


    hashed_password = bcrypt.hashpw(request.password.encode("utf-8"), bcrypt.gensalt())
    hashed_password = hashed_password.decode("utf-8")
    try:
        cursor.execute("INSERT INTO users (firstName, lastName, email, password, role) VALUES (%s, %s, %s, %s, %s)",(request.firstName, request.lastName, request.email, hashed_pass
        print(f"Utilizador com o nome: {request.firstName} foi criado")
    except pymysql.MySQLError:
        raise HTTPException( status_code=500, detail="Erro ao criar utilizador" )

    return {"message": "Utilizador registado com sucesso"}
```

Anexo 7 - Endpoint de Registo

5. Endpoint de Recuperação de Password

Recebe o email, caso não exista irá dar erro Não foi encontrado nenhum email correspondente, caso exista irá criar um token com a biblioteca secrets e um tempo de expiração do token de 10 minutos utilizando o datetime, em seguida a função é chamada para enviar o email e enviar o email para o utilizador caso não haja nenhum erro na criação dos *tokens* e retornará sucesso

```python
@auth_route.post("/recoverPassword")
async def recoverPassword(request: RecoverPassword, background: BackgroundTasks):
    db = get_database()
    cursor = db.cursor()

    cursor.execute("SELECT * FROM users WHERE email = %s", (request.email,))
    user = cursor.fetchone()


    if user is None:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Não foi encontrado nenhum email correspondente")

    token = secrets.token_urlsafe(32)
    expire = datetime.datetime.now() + datetime.timedelta(minutes=10)

    try:
        cursor.execute("UPDATE users SET token = %s, tokenTime = %s WHERE email = %s",(token, expire, request.email))
        background.add_task(send_recovery_email, request.email, token, user["firstName"])
    except pymysql.MySQLError:
        raise HTTPException(status_code=500, detail="Erro ao criar os tokens de verificação")

    return {"message": "Email enviado com sucesso"}
```

Anexo 8 - Endpoint de Recuperação de password

6. Endpoint de Redefinição de Password

Este endpoint acontecerá ao resetar a password após clicar no *link* recebido, irá ser verificado o email e se o token é válido e se o tempo também é válido caso tudo seja válido a password será encriptada com bcrypt e alterada na base de dados e o token será apagado.

```
@auth_route.post("/resetPassword")
async def resetPassword(request:ResetPassword):
    db = get_database()
    cursor = db.cursor()


    cursor.execute("SELECT * FROM users WHERE email = %s", (request.email,))
    user = cursor.fetchone()

    if user == None:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Não foi possível encontrar o email na base de dados")

    if user["token"] != request.token:
        raise HTTPException(status_code=400, detail="Token inválido")

    token_time = datetime.datetime.fromisoformat(str(user["tokenTime"]))

    if token_time < datetime.datetime.now():
        raise HTTPException(status_code=400, detail="Token expirado")

    password = bcrypt.hashpw( request.password.encode("utf-8"), bcrypt.gensalt() ).decode("utf-8")
    cursor.execute( "UPDATE users SET password = %s, token = NULL, tokenTime = NULL WHERE ID = %s", (password, user["ID"]) )

    return{"message": "Password alterada com sucesso"}
```

Anexo 9 - Endpoint de Redefinição de Password

7. Endpoint de Login

Este endpoint procura o email do utilizador na base de dados caso não exista dá erro, caso exista compara com a password enviada com a password da Base de dados encriptada caso esteja correta gera um JWT com: ID, email, role e devolve o token ao *frontend.*

```
@auth_route.post("/login")
async def login(request: LoginRequest):
    db = get_database()
    cursor = db.cursor()

    cursor.execute("SELECT * FROM users WHERE email = %s", (request.email,))
    user = cursor.fetchone()

    if user is None:
        raise HTTPException(status_code=400, detail="Credenciais Inválidas")

    if not bcrypt.checkpw(request.password.encode("utf-8"), user["password"].encode("utf-8")):
        raise HTTPException(status_code=400, detail="Credenciais Inválidas")


    access_token = create_access_token({"sub": str(user["ID"]), "email": user["email"], "role": user["role"]})

    return {
        "access_token": access_token,
        "token_type": "bearer"
    }
```

Anexo 10 - Endpoint de Login

.

Conclusão

O desenvolvimento desta PAP permitiu consolidar conhecimentos técnicos, nomeadamente na criação de APIs, estruturas de dados e como proteger autenticações. ao longo do projeto foram abordados aspetos científicos e tecnológicos, como a utilização do FastAPI, a implementação com *tokens* pelo JWT, a encriptação de passwords com bcrypt, gestão de Base de dados MySQL e a criação de mecanismos de recuperação de credenciais através de *tokens* temporários. A integração com o Cloudflare Tunnel foi igualmente relevante, permitindo disponibilizar a API externamente sem necessidade de se encontrar na mesma rede local, e possibilitou também utilizar esta API com HTTPS tornando ainda mais segura sem necessidade de abrir qualquer porta de algum router e mesmo em ambientes com restrições de rede.

Para além da componente técnica este trabalho contribuiu significativamente para o desenvolvimento de competências profissionais. Devido à necessidade de organização de tarefas, documentar os processos feitos, e até mesmo em avançar com a startup porque este projeto integra-se diretamente no trabalho desenvolvido na startup o que permite um aumento significativo.

Esta PAP representou um desafio que exigiu autonomia, responsabilidade, capacidade de resolução de problemas, persistência e automotivação. A aprendizagem destas ferramentas, até então desconhecidas, representou um desafio que contribuiu significativamente para o meu crescimento,uma vez que a superação de obstáculos promove evolução pessoal e profissional, e este foi um crescimento tanto académico como profissional como pessoal.

Bibliografia

https://fastapi.tiangolo.com/tutorial/

https://www.python.org/doc

https://dev.mysql.com/doc

https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/

https://jwt.io

https://pypi.org/project/bcrypt

https://pymysql.readthedocs.io

https://www.w3schools.com

https://stackoverflow.com

https://github.com

Anexos

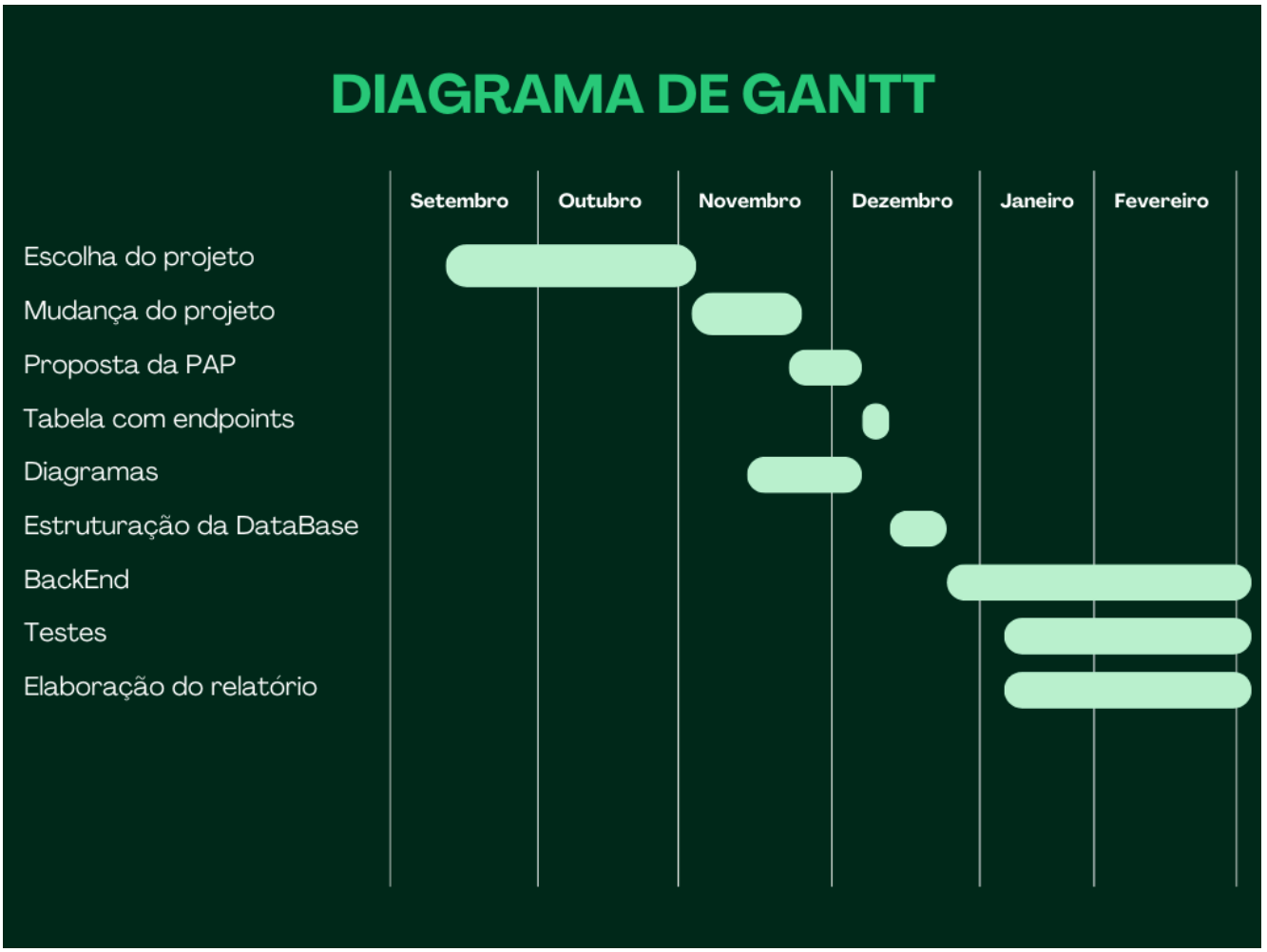
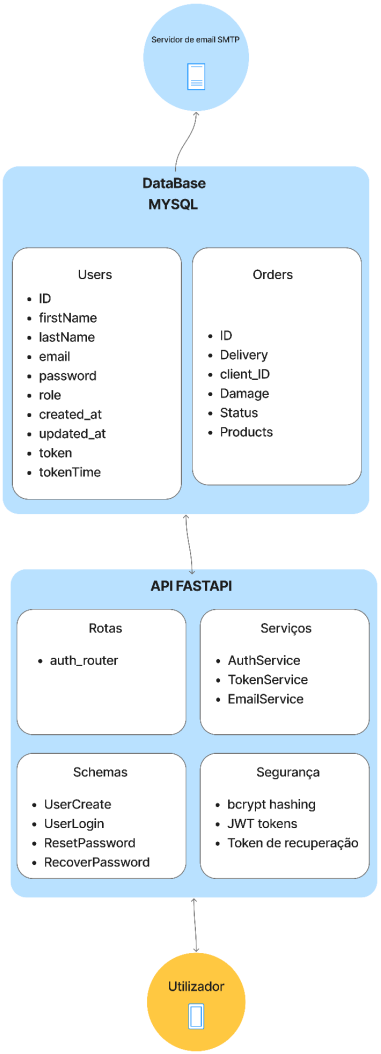


DIAGRAMA DE GANTT
Setembro
Outubro
Novembro
Dezembro
Janeiro
Fevereiro
Escolha do projeto
Mudança do projeto
Proposta da PAP
Tabela com endpoints
Diagramas
Estruturação da DataBase
BackEnd
Testes
Elaboração do relatório

Anexo 1



Anexo 2

Nome	Estado	Data de modificação	Tipo	Tamanho
.venv		06/02/2026 17:22	Pasta de ficheiros	
__pycache__		27/02/2026 18:01	Pasta de ficheiros	
Documentos		01/03/2026 14:42	Pasta de ficheiros	
Relatório		06/02/2026 17:21	Pasta de ficheiros	
.env		26/02/2026 23:01	Ficheiro ENV	1 KB
.gitignore		09/02/2026 08:34	Arquivo Fonte Git ...	1 KB
auth_routes.py		26/02/2026 22:35	Arquivo Fonte Pyt...	5 KB
database.py		22/02/2026 15:32	Arquivo Fonte Pyt...	1 KB
email_recover.py		22/02/2026 15:58	Arquivo Fonte Pyt...	2 KB
jwt_handler.py		26/02/2026 22:06	Arquivo Fonte Pyt...	1 KB
main.py		26/02/2026 10:04	Arquivo Fonte Pyt...	1 KB
requirements.txt		01/03/2026 20:11	Arquivo Fonte Text	3 KB

Anexo 3

```
from fastapi import APIRouter, HTTPException, status, BackgroundTasks
from pydantic import BaseModel
from database import get_database
from email_recover import send_recovery_email
from jwt_handler import create_access_token
import bcrypt
import secrets
import datetime
import pymysql
```

Anexo 4

```
auth_route = APIRouter(prefix="/auth", tags=["Auth"])
```

Anexo 5

```
class RegisterRequest(BaseModel):
    firstName: str
    lastName: str
    email: str
    password: str
    role: str

class LoginRequest(BaseModel):
    email: str
    password: str

class RecoverPassword(BaseModel):
    email: str
class ResetPassword(BaseModel):
    email: str
    password: str
    token: str
```

Anexo 6

```python
@auth_route.post("/register")
async def register(request: RegisterRequest):
    db = get_database()
    cursor = db.cursor()

    cursor.execute("SELECT * FROM users WHERE email = %s", (request.email,))
    user = cursor.fetchone()

    if user:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Email já foi registado")


    hashed_password = bcrypt.hashpw(request.password.encode("utf-8"), bcrypt.gensalt())
    hashed_password = hashed_password.decode("utf-8")
    try:
        cursor.execute("INSERT INTO users (firstName, lastName, email, password, role) VALUES (%s, %s, %s, %s, %s)",(request.firstName, request.lastName, request.email, hashed_password, request.role))
        print(f"Utilizador com o nome: {request.firstName} foi criado")
    except pymysql.MySQLError:
        raise HTTPException( status_code=500, detail="Erro ao criar utilizador" )

    return {"message": "Utilizador registado com sucesso"}
```

Anexo 7

```python
@auth_route.post("/recoverPassword")
async def recoverPassword(request: RecoverPassword, background: BackgroundTasks):
    db = get_database()
    cursor = db.cursor()

    cursor.execute("SELECT * FROM users WHERE email = %s", (request.email,))
    user = cursor.fetchone()


    if user is None:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Não foi encontrado nenhum email correspondente")

    token = secrets.token_urlsafe(32)
    expire = datetime.datetime.now() + datetime.timedelta(minutes=10)

    try:
        cursor.execute("UPDATE users SET token = %s, tokenTime = %s WHERE email = %s",(token, expire, request.email))
        background.add_task(send_recovery_email, request.email, token, user["firstName"])
    except pymysql.MySQLError:
        raise HTTPException(status_code=500, detail="Erro ao criar os tokens de verificação")

    return {"message": "Email enviado com sucesso"}
```

Anexo 8

```python
@auth_route.post("/resetPassword")
async def resetPassword(request:ResetPassword):
    db = get_database()
    cursor = db.cursor()


    cursor.execute("SELECT * FROM users WHERE email = %s", (request.email,))
    user = cursor.fetchone()

    if user == None:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Não foi possível encontrar o email na base de dados")

    if user["token"] != request.token:
        raise HTTPException(status_code=400, detail="Token inválido")

    token_time = datetime.datetime.fromisoformat(str(user["tokenTime"]))

    if token_time < datetime.datetime.now():
        raise HTTPException(status_code=400, detail="Token expirado")

    password = bcrypt.hashpw( request.password.encode("utf-8"), bcrypt.gensalt() ).decode("utf-8")
    cursor.execute( "UPDATE users SET password = %s, token = NULL, tokenTime = NULL WHERE ID = %s", (password, user["ID"]) )

    return{"message": "Password alterada com sucesso"}
```

Anexo 9

```python
@auth_route.post("/login")
async def login(request: LoginRequest):
    db = get_database()
    cursor = db.cursor()

    cursor.execute("SELECT * FROM users WHERE email = %s", (request.email,))
    user = cursor.fetchone()

    if user is None:
        raise HTTPException(status_code=400, detail="Credenciais Inválidas")

    if not bcrypt.checkpw(request.password.encode("utf-8"), user["password"].encode("utf-8")):
        raise HTTPException(status_code=400, detail="Credenciais Inválidas")


    access_token = create_access_token({"sub": str(user["ID"]), "email": user["email"], "role": user["role"]})

    return {
        "access_token": access_token,
        "token_type": "bearer"
    }
```

Anexo 10

main.py > ...

```python

from dotenv import load_dotenv
from fastapi import FastAPI
from auth_routes import auth_route



app = FastAPI()

load_dotenv()



app.include_router(auth_route)


@app.get("/")
async def teste():
    return {"API": "A minha API está a funcionar!"}



```

Anexo 11

jwt_handler.py > ...

```python
from dotenv import load_dotenv
import os
import jwt
from datetime import datetime, timedelta

load_dotenv()

SECRET_KEY = os.getenv("SECRET_KEY")
ALGORITHM = os.getenv("ALGORITHM")

def create_access_token(data: dict, expires_delta: timedelta = timedelta(hours=1)):
    to_encode = data.copy()
    expire = datetime.utcnow() + expires_delta
    to_encode.update({"exp": expire})

    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

```

Anexo 12

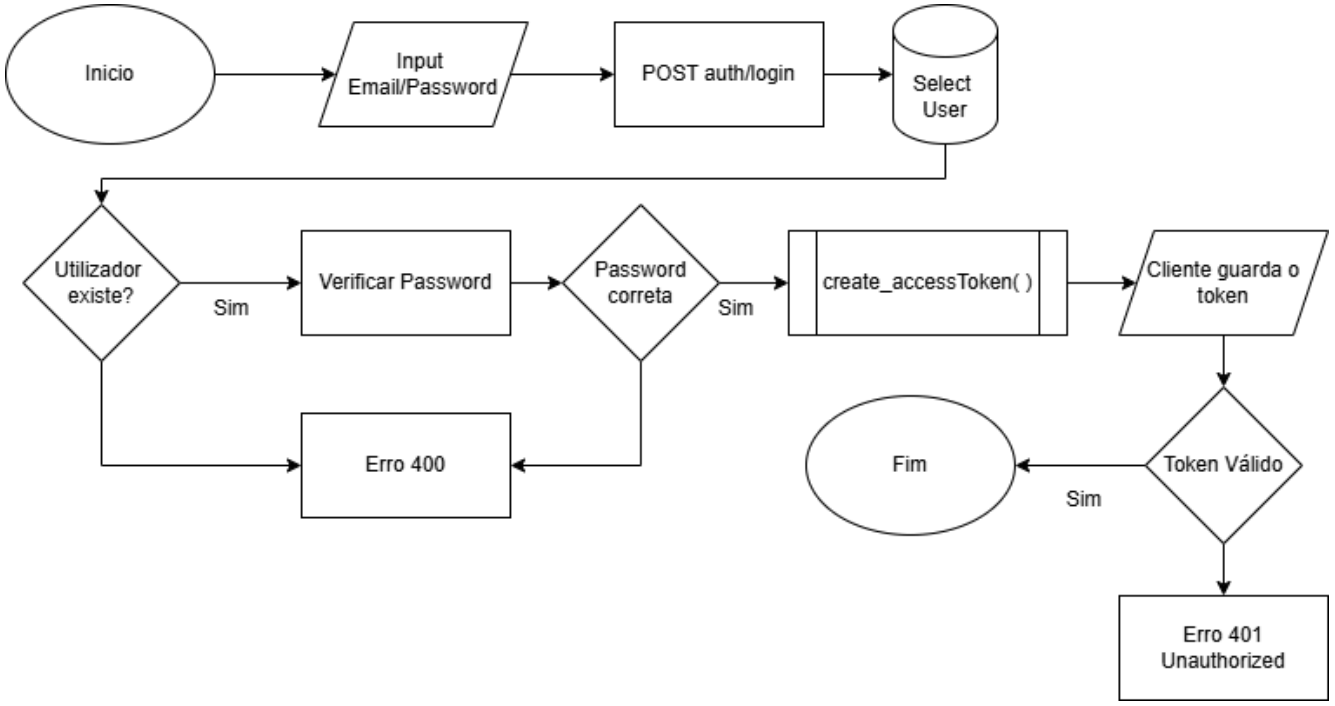
database.py > get_database

```python
import pymysql


print("Script iniciado")

def get_database():
    return pymysql.connect(
    host="127.0.0.1",
    user="root",
    password="123456789",
    database="pap",
    autocommit=True,
    cursorclass=pymysql.cursors.DictCursor
    )



db = get_database()
cursor = db.cursor()


cursor.execute("SELECT * FROM users")
print(cursor.fetchall())
```

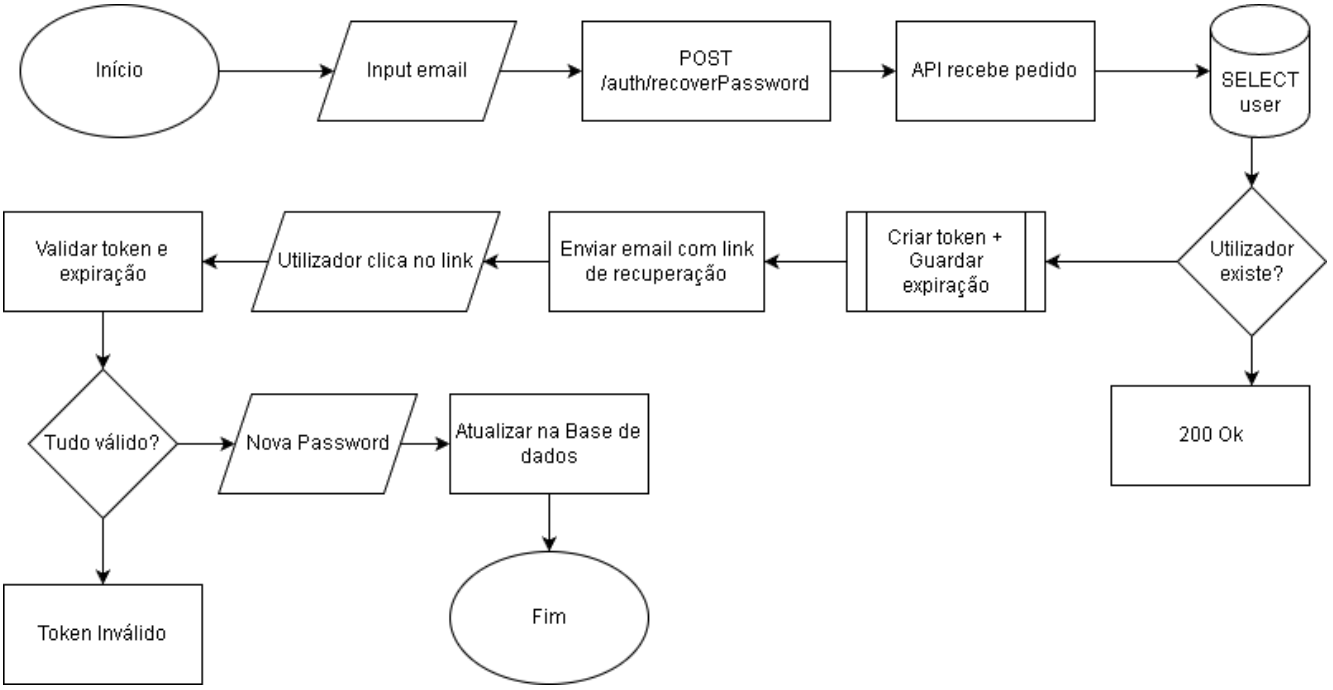
Anexo 13

```python
email_recover.py > send_recovery_email
import os
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

def send_recovery_email(to_email: str, token: str, first_name: str):
    sender_email = os.getenv("EMAIL_USER")
    sender_password = os.getenv("EMAIL_PASS")

    print(f"Email: {sender_email}")
    reset_link = f"http://localhost:3000/reset-password?token={token}"

    subject = "Recuperação de Password"
    body = f'''
    Olá {first_name},

    Este é o suporte da plataforma Solvex, enviaste um pedido para recuperar a tua conta caso não tenhas sido tu a efetuar este pedido, certifica-te que tens acesso á tua conta e altera a palavra-passe caso necessário


    Clica no link abaixo para redefinir a tua password:

    {reset_link}

    '''

    msg = MIMEMultipart()
    msg["From"] = sender_email
    msg["To"] = to_email
    msg["Subject"] = subject
    msg.attach(MIMEText(body, "plain"))

    with smtplib.SMTP("smtp.gmail.com", 587) as server:
        server.starttls()
        server.login(sender_email, sender_password)
        server.sendmail(sender_email, to_email, msg.as_string())
```

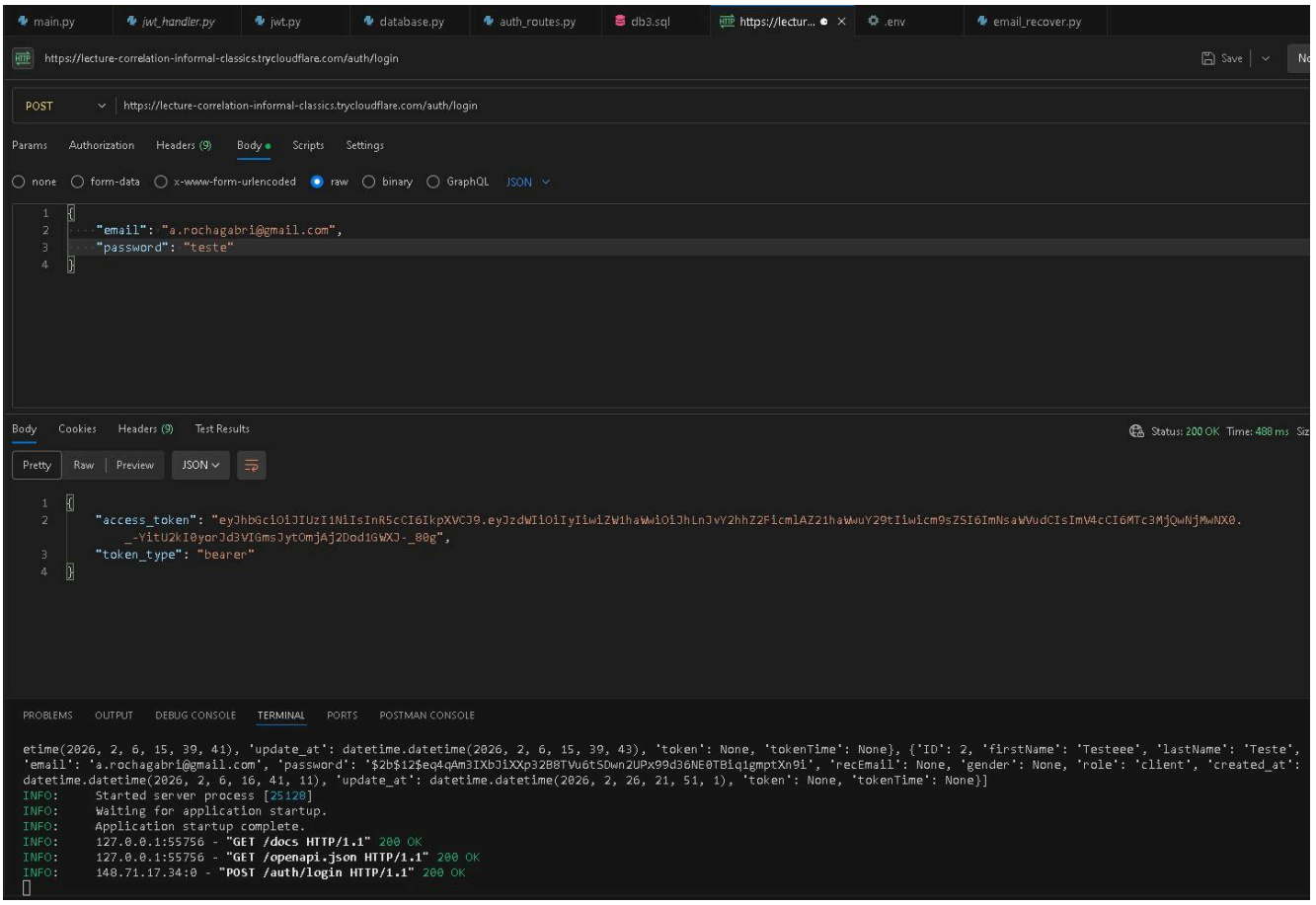
Anexo 14



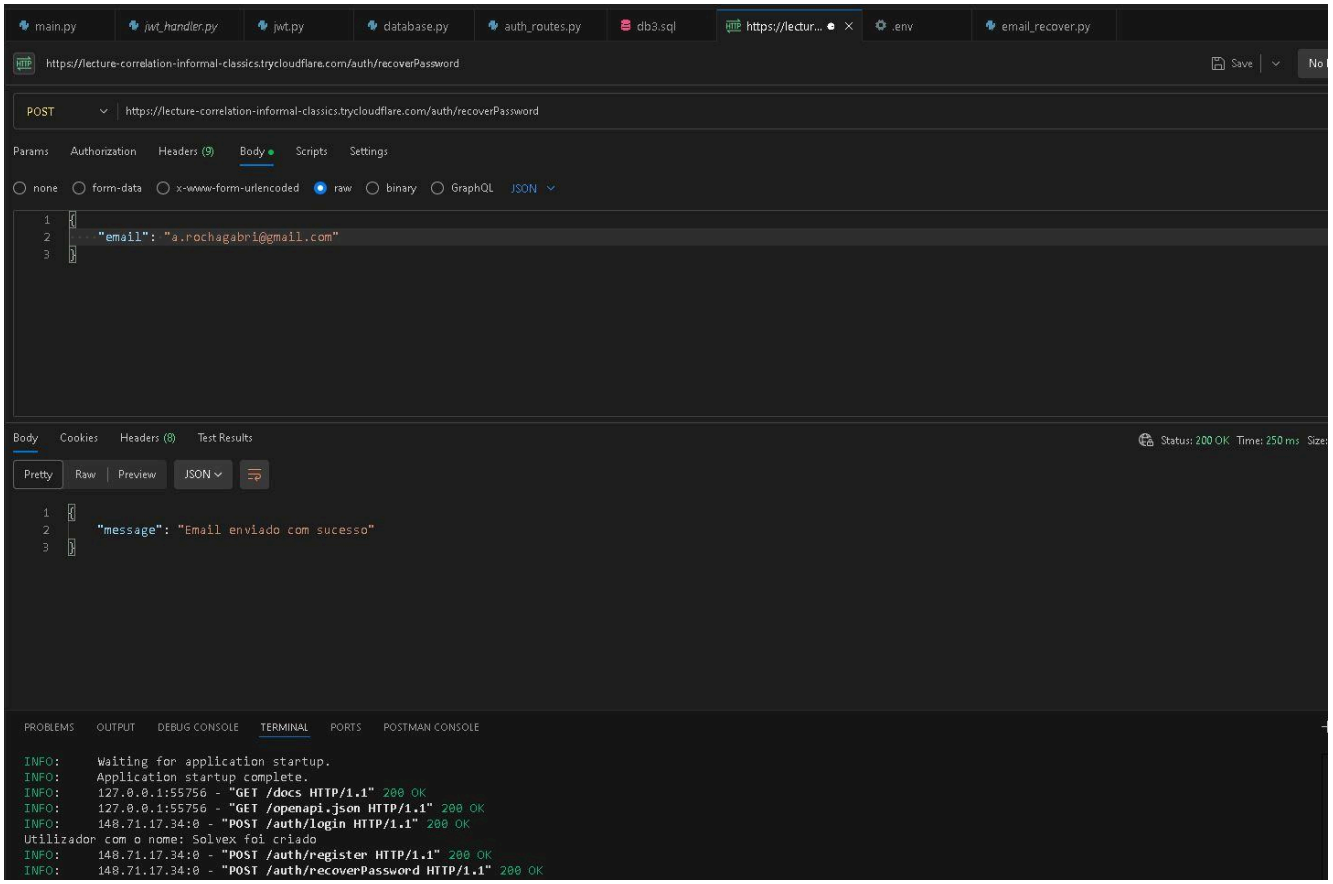
Anexo 15



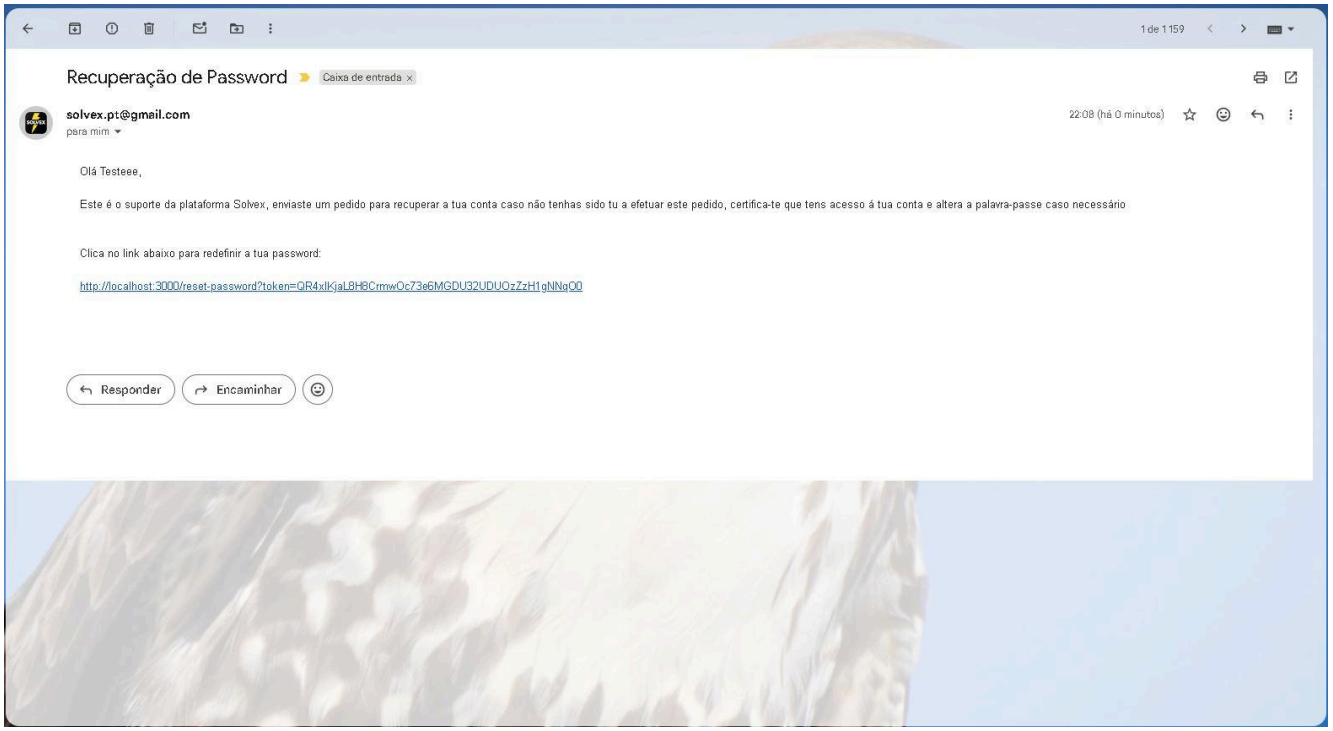
Anexo 16



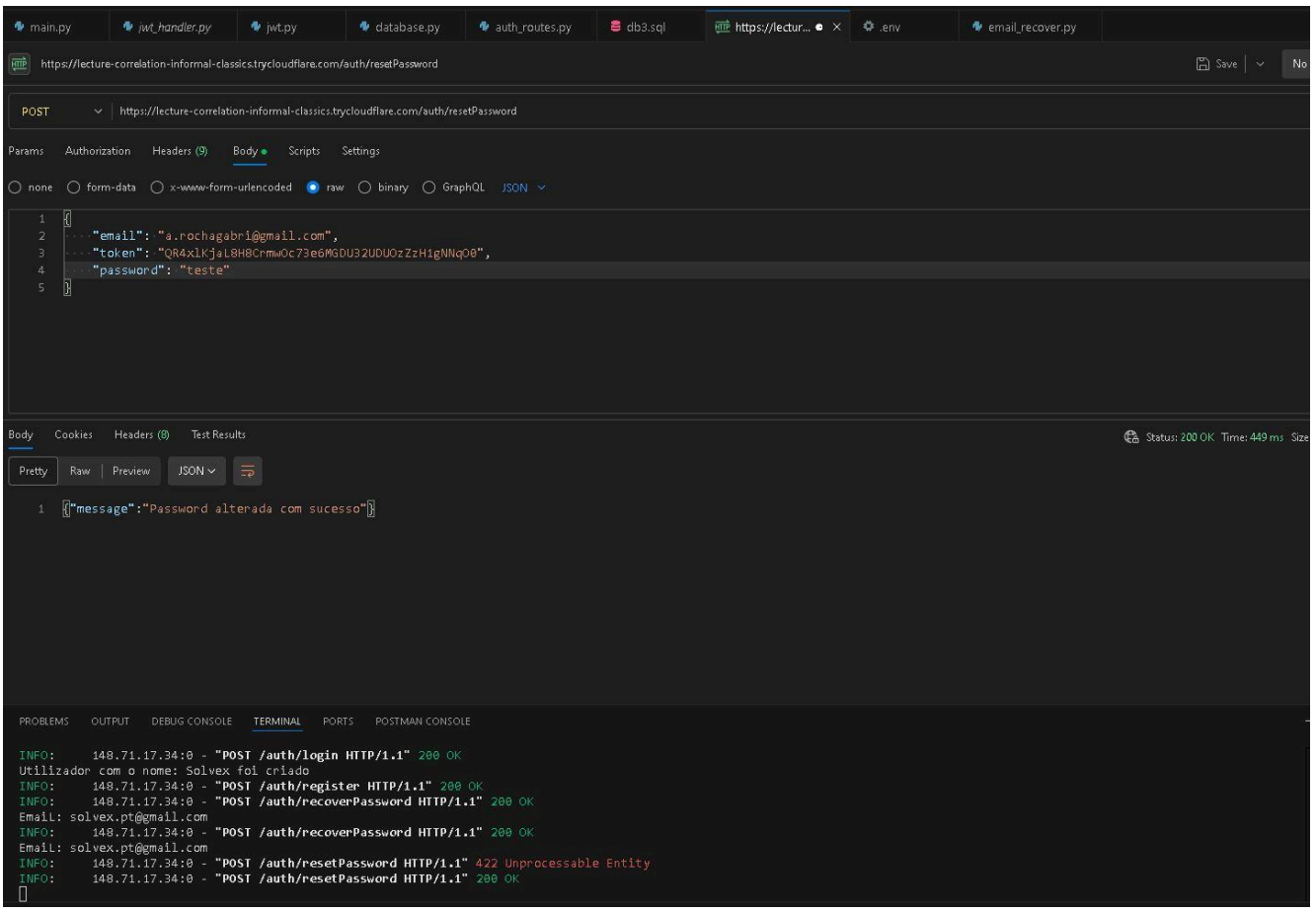
Anexo 17



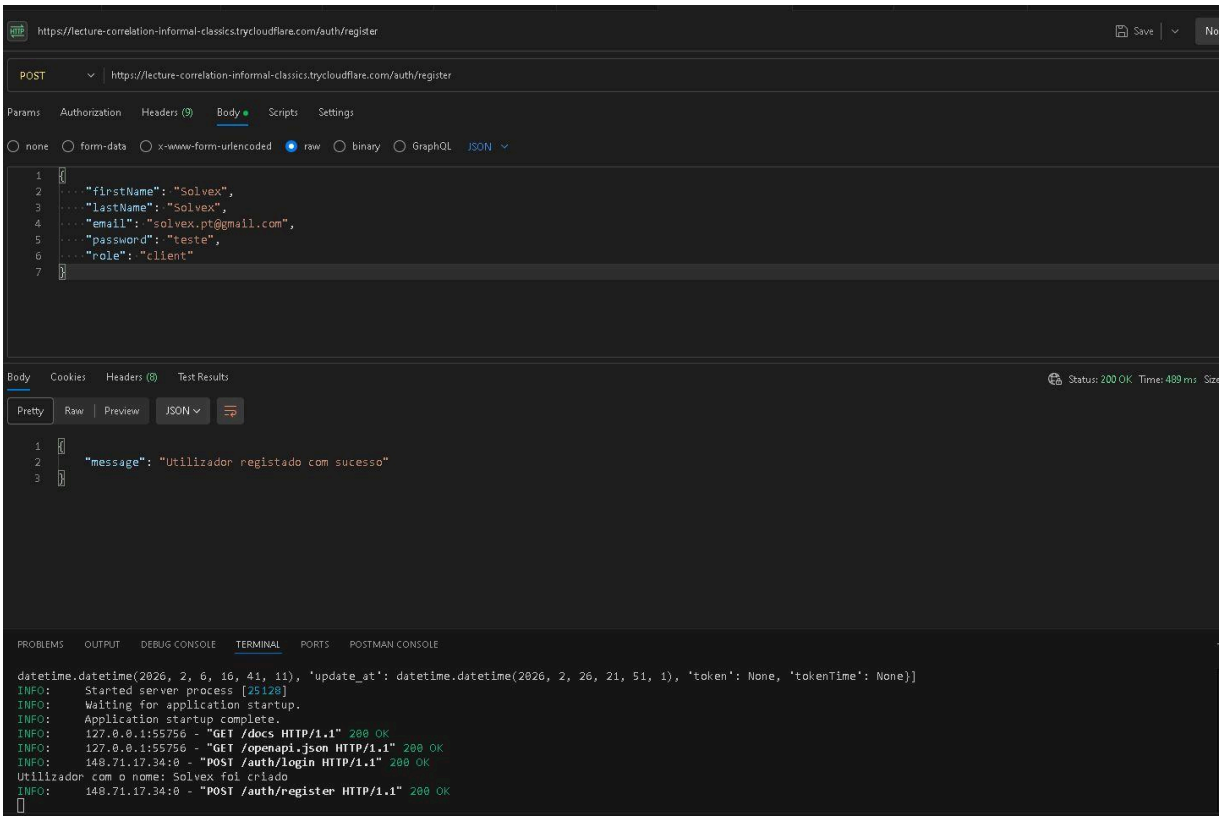
Anexo 18



Anexo 19


main.py
jwt_handler.py
jwt.py
database.py
auth_routes.py
db3.sql
https://lectur...
.env
email_recover.py
https://lecture-correlation-informal-classics.trycloudflare.com/auth/resetPassword
Save
POST
https://lecture-correlation-informal-classics.trycloudflare.com/auth/resetPassword
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"email": "a.rochagabri@gmail.com",
"token": "QR4xlKjaL8H8CrmwOc73e6MGDU32UDUOzZzH1gNNqO0",
"password": "teste"
}
Body
Cookies
Headers (8)
Test Results
Status: 200 OK
Time: 449 ms
Size
Pretty
Raw
Preview
JSON
{"message":"Password alterada com sucesso"}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
INFO: 148.71.17.34:0 - "POST /auth/login HTTP/1.1" 200 OK
Utilizador com o nome: Solvex foi criado
INFO: 148.71.17.34:0 - "POST /auth/register HTTP/1.1" 200 OK
INFO: 148.71.17.34:0 - "POST /auth/recoverPassword HTTP/1.1" 200 OK
Email: solvex.pt@gmail.com
INFO: 148.71.17.34:0 - "POST /auth/recoverPassword HTTP/1.1" 200 OK
Email: solvex.pt@gmail.com
INFO: 148.71.17.34:0 - "POST /auth/resetPassword HTTP/1.1" 422 Unprocessable Entity
INFO: 148.71.17.34:0 - "POST /auth/resetPassword HTTP/1.1" 200 OK

Anexo 20



Anexo 21

ENDPOINTS

EndPoint	Método	Emissor do pedido	Destinatário	Descrição	Corpo do Pedido	Ações Internas importantes
auth/register	Post	Cliente	Server	Regista um novo utilizador	{firstName}, {lastName}, {email}, {password}, {role}	Hash da password com bcrypt
/auth/login	Post	Cliente	Server	Autentica o utilizador e devolve um token JWT	{email}, {password}	Validação da password + geração de JWT
/auth/recoverPassword	Post	Cliente	Server	Gera um token com validade e envia email de recuperação	{email}	Gera token + tokenTime(Duração de 10 minutos)
/auth/resetPassword	Post	Cliente	Server	Valida o token e altera a password	{email}, {password}, {token}	Compara token, verifica tokenTime atualiza a password e limpa o token

Anexo 22